04-4-gates-rationale

accounts-api security assessment · generated from the repository, not hand-edited

> **Supporting write-up for section 4: tool choices, the full-history mechanism, the threshold reasoning, and the expected false-positive load.**
>
> Assembled by `scripts/build-submission.py` from the repository. Source paths are given above each section.

`docs/03-gates-and-waivers.md`

Section 3 — pipeline gates, thresholds, and the waiver path

Artefacts: `.github/workflows/security-gates.yml`, `scripts/waiver_check.py`, `waivers/`. Threats: T1, T2, T3, T4.

Tool choices — one line each

tier	tool	why this one
SCA	**Trivy**	one binary covers deps, OS packages, container images and IaC, so the *same* scanner and the *same* ignore file serve the fs scan, the image scan and the nightly re-scan of shipped digests — fewer semantics for an engineer to hold.
SAST	**Semgrep** (OSS rules + local `policy/semgrep/`)	the only one where I can write a rule for *our* bug in an afternoon; a SAST tool I cannot extend becomes a fixed opinion I have to work around.
secrets	**Gitleaks**	it walks git history via `--log-opts`, which the diff-only scanners structurally cannot; and its allowlist is regex-based, which survives rebases (fingerprints do not).
IaC	**conftest/OPA** blocking, **Checkov** advisory	conftest runs against HCL offline with no `terraform init`, no registry and no cloud credentials, so the merge decision does not depend on network reachability. Checkov brings a large curated ruleset for breadth.
policy tests	**kyverno CLI**, **conftest verify**	the policies are code; untested policy is a comment.

Consolidation, and what I gave up. Trivy owns SCA + container + a second IaC opinion; I did *not* also run Grype and Snyk. What I gave up is Grype's occasionally better Go-module matching and Snyk's reachability analysis — the latter is the real loss, because reachability is exactly what would let me block on fewer, better findings. Compensating: the `not-exploitable` waiver class exists precisely to record a human reachability judgement, with the analysis in `reason` and a 30-day clock on it.

Two tools, one decision. Checkov runs `soft_fail`. Two blocking IaC scanners means two rulesets disagreeing about the same resource and an engineer who cannot tell which one to satisfy — the fastest route to a disabled gate. One tool owns the merge decision.

Full-history secret scanning — the mechanism

Three things have to be true, and the first is the one that is usually missed:

1. `actions/checkout` with **`fetch-depth: 0`**. The default is a depth-1 clone; running a history scan on it exits `0` while examining one commit. That is worse than not running it.
2. `git fetch --unshallow '+refs/heads/*:refs/remotes/origin/*'` — branches deleted on GitHub still have reachable blobs, and a secret is very often on the branch someone deleted after the review comment.
3. `gitleaks detect --log-opts="--all --full-history"`. `gitleaks protect` (the default in most copy-pasted templates) inspects uncommitted changes only.

Cost: on this repo the full walk is seconds. On a ten-year monorepo it is minutes, which is why the full walk is a **push-to-main and nightly** gate while PRs get the fast diff-scoped scan. That gap is deliberate and bounded: a secret can live in a PR branch for the length of the review, but cannot reach `main` unscanned.

Complemented by a `gitleaks protect --staged` pre-commit hook, which is advisory — developer machines are not a control surface.

A finding here is not "rewrite history". Rewriting a shared repo's history is a bad first move: the secret is already in someone's local clone, in GitHub's fork network, and possibly in a cached CI artefact. The order is **rotate → revoke → confirm no use in CloudTrail → then decide about history**, and the workflow prints exactly that on failure.

The threshold that blocks a merge, and why that line

Blocking: CRITICAL and HIGH with a fix available. Warning: everything unfixable, and everything MEDIUM and below.

The interesting half of that sentence is `--ignore-unfixed`. Blocking on a CRITICAL with no upstream patch does not make anyone safer; it makes the only available action "file a waiver". Ten of those and the team has learned that the gate is a paperwork generator, and the next real CRITICAL gets waived with the same reflex. So the blocking set is deliberately *the set of findings a developer can actually act on today*, and the unfixable backlog goes to a non-blocking scan that is tracked and trended instead.

Two deliberate exceptions to severity-based thresholds:

- **Secrets block at any severity.** Severity is meaningless for a credential; the only question is whether it is live, and the answer is assumed yes.
- **IaC blocks on rule identity, not severity.** `policy/rego/terraform.rego` only contains rules about data-plane exposure — public database, unencrypted storage, world-open security group, disabled audit trail. Every one of those is a "block", because the scanner's own severity score has no idea that this particular RDS instance holds IBANs.

Promotion path for the WARN gates

SAST ships in WARN. The promotion criterion is not a date, it is **evidence**:

- two consecutive weeks with **zero** new ERROR-severity findings on `main` that are subsequently triaged as false positives, and
- the standing backlog is under ten findings, each with an owner, and
- the median PR-to-green time has not moved by more than 20%.

When those hold, the `|| true` comes out in a PR that a security lead reviews, and the date in the workflow's warning text is the announced deadline (currently 2026-10-15). If the evidence does not hold by that date, the *rules* get cut down — the gate still promotes, on a smaller ruleset. A gate that never promotes is a gate that was never a gate. Full posture table in `../DECISIONS.md` §2.

Waiver mechanism

Specified and implemented; the lifecycle, approver matrix, expiry behaviour and false-positive tuning order are in **`../waivers/README.md`**. Two properties worth repeating here because they are the ones that make it more than a `.trivyignore` with a comment:

- The suppression files are **generated, gitignored and derived from unexpired waivers only** — so expiry is not a reminder, it is a mechanism.
- The approval is **signed over a canonical body**, so `expires` cannot be quietly extended. `waivers/fixtures/tampered.yaml` demonstrates the failure.

Break-glass — the one path that bypasses the blocking gates — is in `03-break-glass.md`.